🎓 QuizMind-Ai (Classroom Quiz Platform) — Complete Interview Preparation Guide

This document is a comprehensive, structured interview preparation guide covering all core technical, architectural, AI, database, security, and workflow aspects of **QuizMind-Ai**.

📋 Table of Contents

1. Project Overview & Problem Statement
2. Project Architecture & Complete User Workflow
3. Technology Stack & Architectural Rationale
4. AI Integration & Dynamic Question Generation
5. Quiz Execution, Auto-Grading & Data Storage
6. REST API Flow, Security & JWT Authentication
7. Key Technical Optimizations & Interview Talking Points

---

1. Project Overview & Problem Statement

Q1: Tell me about your Quiz Minds AI project.

Answer:
QuizMind-Ai is an end-to-end, enterprise-ready classroom quiz and assessment backend platform built using **Java 21**, **Spring Boot 3.2.2**, **Spring Data MongoDB**, and **Spring AI** powered by local **Ollama LLMs (qwen2.5-coder:1.5b)**.

It provides a dual-role ecosystem:
- **Instructors/Admins**: Can manually create quizzes, configure time limits, schedule live exam sessions, assign sessions to specific students, and view real-time leaderboards.
- **Students**: Can participate in live scheduled quizzes, take dynamic AI-generated practice tests on demand on any topic, attempt questions with per-question time limits, and receive instant automated grading with negative marking.

Q2: Why did you develop this project?

Answer:
Traditional educational assessment platforms suffer from three major bottlenecks:
1. **Time-Consuming Quiz Creation**: Teachers spend hours manually composing multiple-choice questions (MCQs), options, and answers.
2. **Lack of Instant Adaptive Practice**: Students preparing for exams lack instant, custom-tailored practice tests on specific topics without teacher intervention.

3. **Manual Evaluation & Scalability Bottlenecks**: Grading high-volume classroom assessments manually is slow and prone to error, especially when implementing negative marking or timed per-question constraints.

We developed **QuizMind-Ai** to automate the end-to-end lifecycle of quiz creation, delivery, AI-assisted practice, auto-evaluation, and scheduled auto-submission.

Q3: What problem does Quiz Minds AI solve?

Answer:
- **Instant AI Practice Generation**: Converts any subject/topic query into a fully structured, 4-option MCQ quiz in seconds using local LLM inference via Spring AI.
- **Automated Anti-Cheating & Session Management**: Enforces strict scheduled session start/end windows, prevents duplicate attempts, and tracks per-question time limits.
- **Background Auto-Submission**: Eliminates lost student submissions by using a background `@Scheduled` service that automatically finalizes and grades expired student attempts.
- **High Concurrency & Database Efficiency**: Leverages MongoDB's document model to store quizzes, questions, and attempts with optimized single-query batch reads ($N+1$ optimization).

---

2. Project Architecture & Complete User Workflow

Q4: Explain the complete workflow of your project from user login to quiz submission.

Answer:

```
sequenceDiagram
    autonumber
    actor User as Student/Admin
    participant Auth as Auth Controller
    participant Filter as JwtAuthenticationFilter
    participant AI as Spring AI / Ollama
    participant QuizService as Quiz / Attempt Service
    participant DB as MongoDB

    User->>Auth: POST /auth/login (email, password)
    Auth->>DB: Verify credentials & BCrypt password
    Auth-->>User: Return JWT Token (with Role & UserID claims)

    User->>AI: POST /api/ai/generate-and-start?topic=Java&count=5 (Bearer JWT)
    Filter->>Filter: Validate JWT in-memory (Extract UserID & Role)
    AI->>AI: Prompt Ollama LLM (qwen2.5-coder:1.5b)
    AI->>DB: Save Quiz, Questions & QuizSession
    AI-->>User: Return SessionID

    User->>QuizService: POST /student/attempts/start/{sessionId}
    QuizService->>DB: Verify window & create Attempt document
    QuizService-->>User: Return AttemptID

    User->>QuizService: GET /student/quizzes/{quizId}
    QuizService-->>User: Return Quiz questions & options

    User->>QuizService: POST /student/attempts/answer (Selected Option)
    QuizService->>DB: Verify per-question time limit & save answer in map

    User->>QuizService: POST /student/attempts/submit/{attemptId}
    QuizService->>DB: Grade answers (Correct: +4, Incorrect: -1) & save score
    QuizService-->>User: Return Final Score
```

Step-by-Step Workflow:

1. **Authentication**: User logs in at `/auth/login`. System validates password using `BCryptPasswordEncoder` and generates a signed JWT token containing `userId` and `role`.
2. **AI Quiz Generation**: Student requests a practice quiz via `POST /api/ai/generate-and-start?topic=Java&count=5`. The `QuizAiService` calls Ollama, parses the JSON response, persists `Quiz`, `Question`, and `QuizSession` documents in MongoDB, and returns a `sessionId`.
3. **Starting Attempt**: Student initiates test via `POST /student/attempts/start/{sessionId}`. System checks session validity, verifies student permission, ensures no prior attempt exists, and creates an `Attempt` record.
4. **Fetching Quiz Data**: Student fetches questions using `GET /student/quizzes/{quizId}`.
5. **Answering Questions**: As the student selects answers, `POST /student/attempts/answer` records selected indices in the `attempt.answers` map while enforcing per-question time limits.
6. **Quiz Submission & Auto-Grading**: `POST /student/attempts/submit/{attemptId}` calculates marks (+4 for correct, -1 for incorrect), updates final score, marks `submitted = true`, and returns score.

Q5: Explain the architecture of your project.

Answer:
QuizMind-Ai follows a **Layered Architecture (Controller-Service-Repository)** with a decoupled Security Layer and asynchronous background scheduling:



Q6: What was your specific contribution to the project?

Answer:
- **Architected the Core Spring Boot Backend**: Designed the domain models (`User` , `Quiz` , `QuizSession` , `Question` , `Attempt`) and repository interfaces.
- **Implemented Spring AI Integration**: Integrated Spring AI with local Ollama LLM (`qwen2.5-coder:1.5b`) using custom prompt engineering (`AiPromptBuilder`) for deterministic JSON extraction.
- **Built JWT Authentication & Security Filter**: Created `JwtAuthenticationFilter` with an **In-Memory JWT Claims Bypass Optimization** to eliminate database queries on every authenticated API request.
- **Engineered Auto-Grading & Performance Optimizations**: Solved $N+1$ query overhead in attempt grading by batch-loading questions using `findByQuizId` into an in-memory hash map.
- **Implemented Background Auto-Submission**: Created `AutoSubmitService` with Spring `@Scheduled` tasks to auto-close expired quiz attempts every 30 seconds.

---

3. Technology Stack & Architectural Rationale

Q7: What technologies did you use and why did you choose them?

Answer:

Technology	Purpose	Reason for Selection
Java 21	Primary Programming Language	Pattern matching, record types, enhanced performance, long-term support (LTS).
Spring Boot 3.2.2	Backend Framework	Rapid API development, auto-configuration, robust dependency injection, production readiness.
Spring Data MongoDB	Database Access	Seamless object-document mapping (ODM), dynamic schema handling for quizzes/answers.
MongoDB	NoSQL Database	Flexible document model for embedded question IDs and key-value answer maps.
Spring AI (Ollama Starter)	AI Abstraction Layer	Standardized ChatClient API for local LLM integration without hardcoded HTTP clients.
Ollama (`qwen2.5-coder:1.5b`)	Local LLM Engine	Fast, privacy-preserving local AI generation without external API costs or latency.
Spring Security + JWT (`jjwt 0.11.5`)	Authentication & Authorization	Stateless, secure, role-based authorization (`ROLE_ADMIN` , `ROLE_STUDENT`).
Lombok & MapStruct	Code Cleanliness & DTO Mapping	Boilerplate reduction and compile-time type-safe object mapping.

Q8: Why did you choose Spring Boot for the backend?

Answer:
1. **Enterprise Reliability & Ecosystem**: Spring Boot provides out-of-the-box integration with Spring Security, Spring Data, and Spring AI.
2. **Stateless Scalability**: Effortlessly handles stateless JWT session management and REST API design.
3. **Production-Ready Features**: Built-in exception handling (`@RestControllerAdvice`), validation (`@Valid`), and openAPI documentation (`springdoc-openapi`).

Q9: Why did you choose MongoDB?

Answer:
1. **Flexible Schema for Quiz Data**: Quizzes and attempts contain nested, non-uniform structures (e.g., dynamic key-value maps for `answers` , variable option array lengths, and flexible categories).
2. **JSON-Native Storage**: Questions generated by AI are formatted as JSON arrays, mapping naturally to MongoDB BSON documents without complex SQL table joins.

3. **High Read Throughput**: MongoDB excels at document lookup by ID (`findById`), enabling sub-millisecond retrieval of user attempts and quizzes.

Q10: Why did you use Spring AI?

Answer:
1. **Unified AI Interface**: Spring AI provides the `ChatClient` abstraction, allowing us to swap underlying AI providers (Ollama, OpenAI, Anthropic) with zero logic changes.
2. **Local LLM Integration**: Native support for Ollama allows running `qwen2.5-coder:1.5b` locally on `http://localhost:11434` , ensuring zero API costs, full privacy, and offline capabilities.

---

4. AI Integration & Dynamic Question Generation

Q11: How does the application generate questions using AI?

Answer:
1. **Prompt Construction**: `AiPromptBuilder.buildQuizPrompt(topic, count)` generates a prompt with strict formatting rules requiring the model to return **ONLY a JSON array** without markdown or conversational commentary.
2. **Spring AI Execution**: `ChatClient` sends the prompt to Ollama (`qwen2.5-coder:1.5b`).
3. **Clean Extraction & Parsing**: `QuizAiService.parseResponse()` strips markdown block backticks (```` ```json ````), locates array boundaries (`[` and `]`), and deserializes the JSON string into `List<AiQuestionResponse>` using Jackson `ObjectMapper` .
4. **Database Persistence**: Converts `AiQuestionResponse` objects into `Question` entities, saves them via `QuestionRepository.saveAll()` , creates a `Quiz` linking these questions, and initializes a `QuizSession` .

Q12: What happens when a user clicks "Generate Quiz"?

Answer:
1. API receives request: `POST /api/ai/generate-and-start?topic=Java&count=5` .
2. Input validation checks if `topic` is non-empty and `count` is between 1 and 10.
3. Spring AI requests 5 questions from local Ollama LLM.
4. System creates and saves a `Quiz` entity with `duration = count * 2` minutes, `marksPerQuestion = 4` , and `negativeMarks = 1` .
5. System creates 5 `Question` entities in MongoDB.
6. System creates a `QuizSession` restricted to the requesting `studentId` .
7. Endpoint returns the newly created `sessionId` directly to the user.

---

5. Quiz Execution, Auto-Grading & Data Storage

Q13: What happens when a user submits a quiz?

Answer:
When `POST /student/attempts/submit/{attemptId}` is called:
1. Verifies that the attempt exists and has not already been submitted (`submitted == false`).
2. Retrieves the associated `Quiz` document to obtain `marksPerQuestion` (4) and `negativeMarks` (1).
3. **Batch Question Retrieval**: Fetches all questions for the quiz in a single database call (`questionRepository.findByQuizId(...)`) and maps them by ID.
4. **Answer Evaluation Loop**:
- Compares the student's selected option index (`entry.getValue()`) with the question's `correctAnswerIndex` .
- If correct: `score += marksPerQuestion` (+4).
- If incorrect: `score -= negativeMarks` (-1).
5. **Floor Enforcement**: If `score < 0` , sets `score = 0` .
6. **State Persistence**: Sets `attempt.setSubmitted(true)` , `submittedAt = Instant.now()` , saves the attempt to MongoDB, and returns the final score.

Q14: How is the user's score calculated?

Answer:
$$\text{Final Score} = \max\left(0, \sum_{i \in \text{Answered}} \text{Grading}(i)\right)$$

$$\text{Grading}(i) = \begin{cases} +4 & \text{if } \text{SelectedOption}_i = \text{CorrectOption}_i \ -1 & \text{if } \text{SelectedOption}_i \neq \text{CorrectOption}_i \end{cases}$$

Unanswered questions receive 0 marks.

Q15: How is the quiz/result data stored in MongoDB?

Answer:

Collections & Schemas:

```
// Collection: quizzes
{
  "_id": "6aae272ec5288b0521f4e3b6",
  "title": "AI Practice: Java",
  "topic": "Java",
  "duration": 4,
  "marksPerQuestion": 4,
  "negativeMarks": 1,
  "totalMarks": 8,
  "questionIds": ["6aae272ec5288b0521f4e3b7", "6aae272ec5288b0521f4e3b8"],
  "active": true
}

// Collection: questions
{
  "_id": "6aae272ec5288b0521f4e3b7",
  "quizId": "6aae272ec5288b0521f4e3b6",
  "questionText": "What is JVM?",
  "options": ["Java Virtual Machine", "Java Version Manager", "Joint Vendor Mode", "None"],
  "correctAnswerIndex": 0,
  "category": "AI-Generated",
  "timeLimitSeconds": 60
}

// Collection: attempts
{
  "_id": "6aae274fc5288b0521f4e3ba",
  "sessionId": "6aae272ec5288b0521f4e3b9",
  "quizId": "6aae272ec5288b0521f4e3b6",
  "studentId": "6aae26dc3c5288b0521f4e3b4",
  "answers": {
    "6aae272ec5288b0521f4e3b7": 0
  },
  "score": 4,
  "submitted": true,
  "startedAt": "2026-09-19T06:10:15Z",
  "submittedAt": "2026-09-19T06:11:00Z"
}
```

---

6. REST API Flow, Security & JWT Authentication

Q16: Explain the REST API flow in your project.

Answer:

```
[ Client Request ]
       |
       v
[ JwtAuthenticationFilter ] ---> Validates Token -> Populates SecurityContextHolder
       |
       v
[ SecurityConfig Rules ] ---> Checks Authority (e.g. ROLE_STUDENT / ROLE_ADMIN)
       |
       v
[ RestController ] ---> Validates Payload (@Valid)
       |
       v
[ Service Layer ] ---> Business Logic & Transactions
       |
       v
[ Repository Layer ] ---> MongoDB ODM (Spring Data)
       |
       v
[ JSON Response ] <--- HTTP Status 200 / 201 / 400 / 403 / 500
```

Q17: How did you implement authentication?

Answer:
1. **Registration**: `/auth/register` validates input (`@Valid`), encodes passwords using `BCryptPasswordEncoder` , sets default role (`ADMIN` or `STUDENT`), and saves `User` document.
2. **Login**: `/auth/login` uses `AuthenticationManager` to authenticate credentials against `CustomUserDetailsService` .
3. **Token Generation**: Upon successful authentication, `JwtTokenProvider.generateToken()` generates a signed HS256 JWT containing `userId` as the subject and user `role` as a custom claim.

Q18: Why did you use JWT?

Answer:
1. **Stateless Server Architecture**: No server-side session storage or memory overhead (Redis/HTTP sessions), allowing seamless horizontal scaling.
2. **Decentralized Authorization**: The server validates requests by cryptographically verifying the signature of the incoming JWT token.
3. **Cross-Origin Compatibility**: Works seamlessly with modern web clients, mobile apps, and microservices via standard `Authorization: Bearer <token>` HTTP headers.

Q19: Explain the complete JWT authentication workflow.

Answer:

```
1. Client sends POST /auth/login with credentials.
2. Server verifies credentials & generates JWT signed with HMAC-SHA256 key:
   Header:  {"alg": "HS256", "typ": "JWT"}
   Payload: {"sub": "<userId>", "role": "ROLE_STUDENT", "iat": 1789798110, "exp": 1789801710}
   Signature: HMACSHA256(header + payload, secretKey)
3. Client receives token and attaches it to subsequent headers:
   Authorization: Bearer <jwt_token>
4. JwtAuthenticationFilter intercept requests:
   - Extracts string after "Bearer "
   - Parses & verifies signature with signingKey
   - Extracts userId & role
   - Constructs CustomUserDetails in memory (NO DB QUERY!)
   - Sets UsernamePasswordAuthenticationToken into SecurityContextHolder
```

Q20: How did you secure your APIs?

Answer:
1. **Spring Security Rule Chain**:
- `/auth/**`, `/swagger-ui/**`, `/v3/api-docs/**` -> Permitted to all (`permitAll()`).
- `/admin/**` -> Restricted to `ROLE_ADMIN` / `ADMIN`.
- `/student/**` & `/api/ai/**` -> Restricted to `ROLE_STUDENT` / `STUDENT`.
- All other routes require authentication.
2. **Method-Level Security**: Enabled `@EnableMethodSecurity` and `@PreAuthorize("hasAnyAuthority(...)")` on controller methods.
3. **Password Hashing**: BCrypt hashing with salt.
4. **CORS Configuration**: Explicit `CorsConfigurationSource` allowing custom origins, HTTP methods (`GET`, `POST`, `PUT`, `DELETE`), and `Authorization` headers.

---

7. Key Technical Optimizations & Interview Talking Points

🌟 Top 3 Technical Achievements to Mention in Interviews:

1. **DB Bypass Optimization in JWT Filter**:

> *"Instead of querying MongoDB on every single HTTP request to fetch user details, our `JwtAuthenticationFilter` reads the `userId` and `role` claims directly from the cryptographically signed JWT payload and constructs the `CustomUserDetails` principal in-memory. This reduced database round-trips by over 90% for authenticated API requests."*

2. **$N+1$ Database Optimization during Quiz Grading**:

> *"When a student submits a quiz, naive grading loops perform a DB lookup for every answered question. I optimized this by batch-fetching all quiz questions in a single query (`findByQuizId`) and storing them in an in-memory hash map (`Map<String, Question>`), turning $N+1$ queries into $O(1)$ map lookups."*

3. **Background Auto-Submission via Spring Scheduling**:

> *"To solve the issue of unsubmitted quizzes when student session times expire, I implemented an asynchronous `AutoSubmitService` with `@Scheduled(fixedRate = 30000)` that periodically fetches expired, unsubmitted attempts and triggers auto-grading automatically."*
